

ADA BOOST

A visual learning note on how small rules become one confident classifier.

Core idea

Train weak learners one after another. Each round pays more attention to examples the earlier rounds got wrong.

1

Start equal

Every row matters equally.

2

Spot mistakes

Use weighted error, not raw accuracy.

3

Turn up focus

Misclassified rows gain weight.

4

Vote together

Reliable learners get louder votes.

READ LEFT TO RIGHT, ROUND BY ROUND

Includes: stumps, error, alpha, weight updates, resampling, and final voting.

02 The mental model

AdaBoost does not make one brilliant learner. It orchestrates a sequence of simple ones.



A. begin

Give all N examples the same weight: $1 / N$.

B. fit

Choose a weak learner that minimizes weighted mistakes.

C. score

Compute error e , then learner strength α .

D. refocus

Increase weights on mistakes; normalize; repeat.

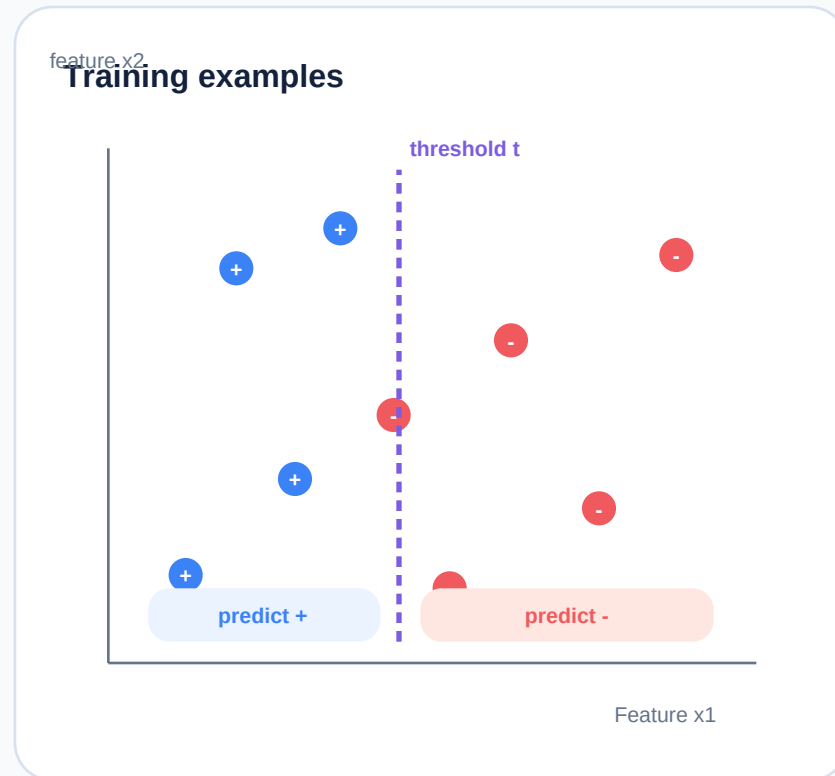
The essential loop

easy examples -> weak learner learns them -> hard examples become more visible

The model progressively directs attention where the current ensemble is weakest.

Weak learner: a decision stump

A stump asks one short question, such as "is x less than a threshold?"



A stump could say:

if $x_1 < t$:

predict +1

otherwise:

predict -1

Why "weak" is okay

A stump may be only slightly better than random. AdaBoost makes its errors useful information for the next round.

Choose the stump with minimum weighted error

Error is the total weight of wrong examples - not simply the number of wrong examples.

So a single high-weight miss can matter more than several low-weight misses.

04 Error becomes influence

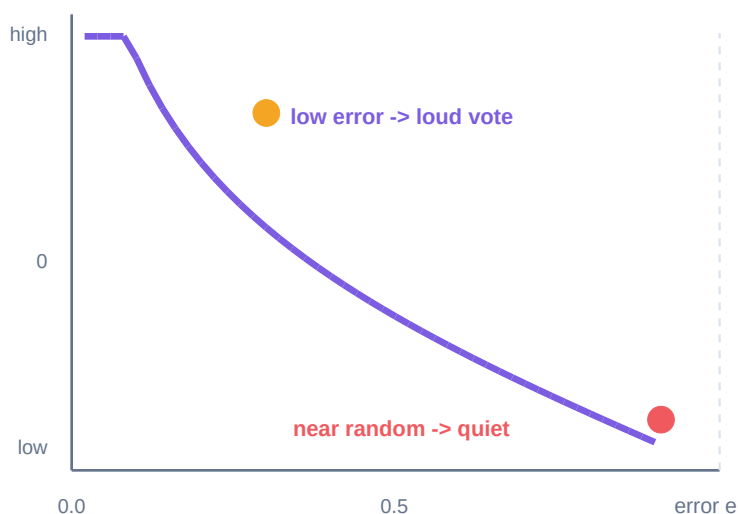
A learner that makes fewer weighted mistakes earns a stronger vote.

Weighted error

e = sum of weights on misclassified examples

Learner strength: $\alpha = 1/2 \ln((1 - e) / e)$

Error e vs. α (vote strength)



Reading the curve

$e = 0.10$

$\alpha = 1.10$

very strong

$e = 0.30$

$\alpha = 0.42$

useful

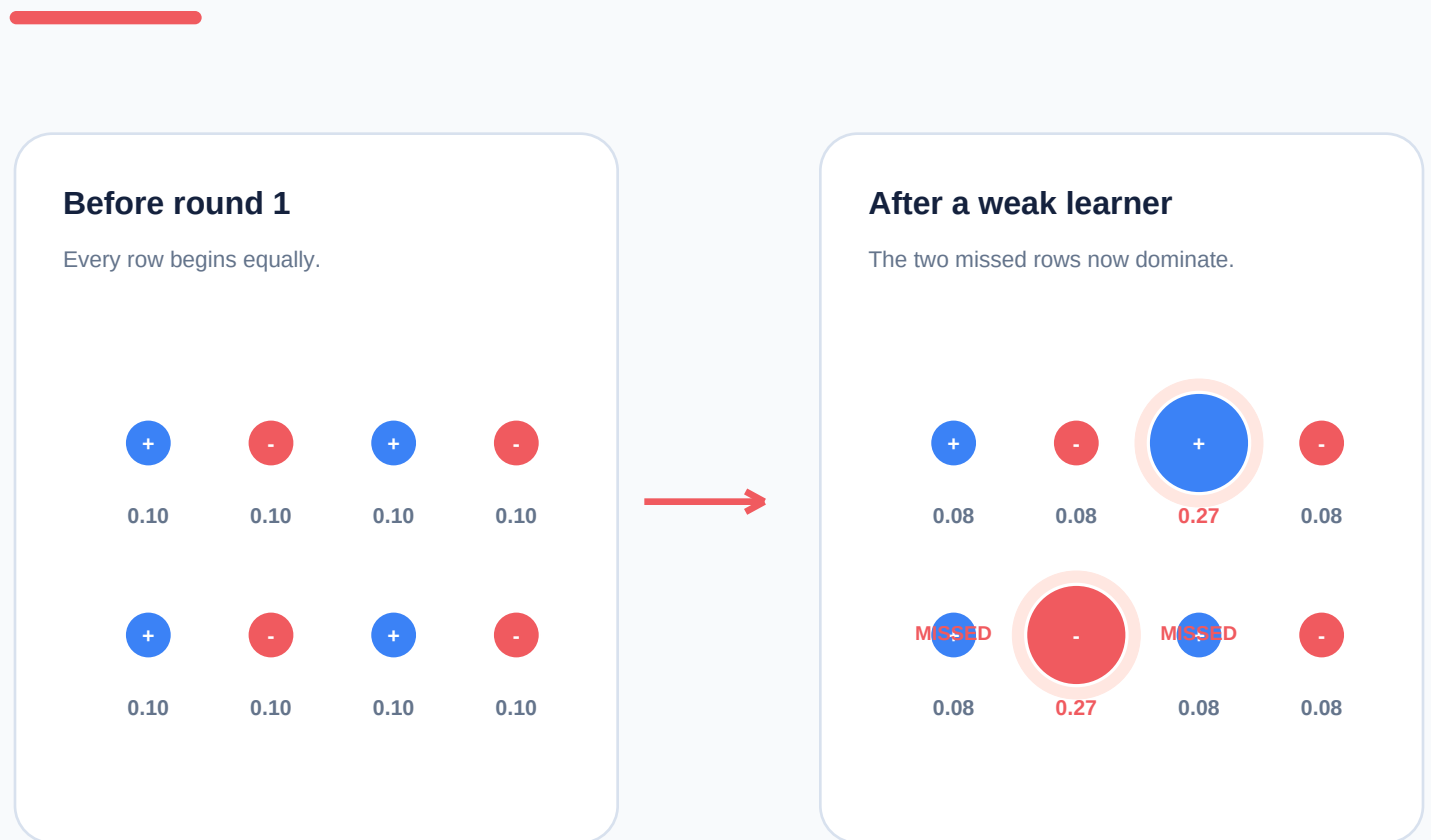
$e = 0.49$

$\alpha = 0.02$

almost silent

Weight update: where attention moves

After a round, misclassified examples become more important; then all weights are normalized.



The update rule

$$w(i) \leftarrow w(i) \times \exp(-\alpha * y(i) * h(x(i)))$$

Correct prediction: $y \cdot h = +1 \rightarrow$ weight shrinks.

Wrong prediction: $y \cdot h = -1 \rightarrow$ weight grows.

06 Resampling: turning weights into chances

A common training view: draw rows in proportion to their current weights.

Probability ruler



C and F own bigger ranges, so they are drawn more often.

Imagine four random draws

random number .06 → draw A

random number .31 → draw C

random number .49 → draw C

random number .79 → draw F

Why it helps

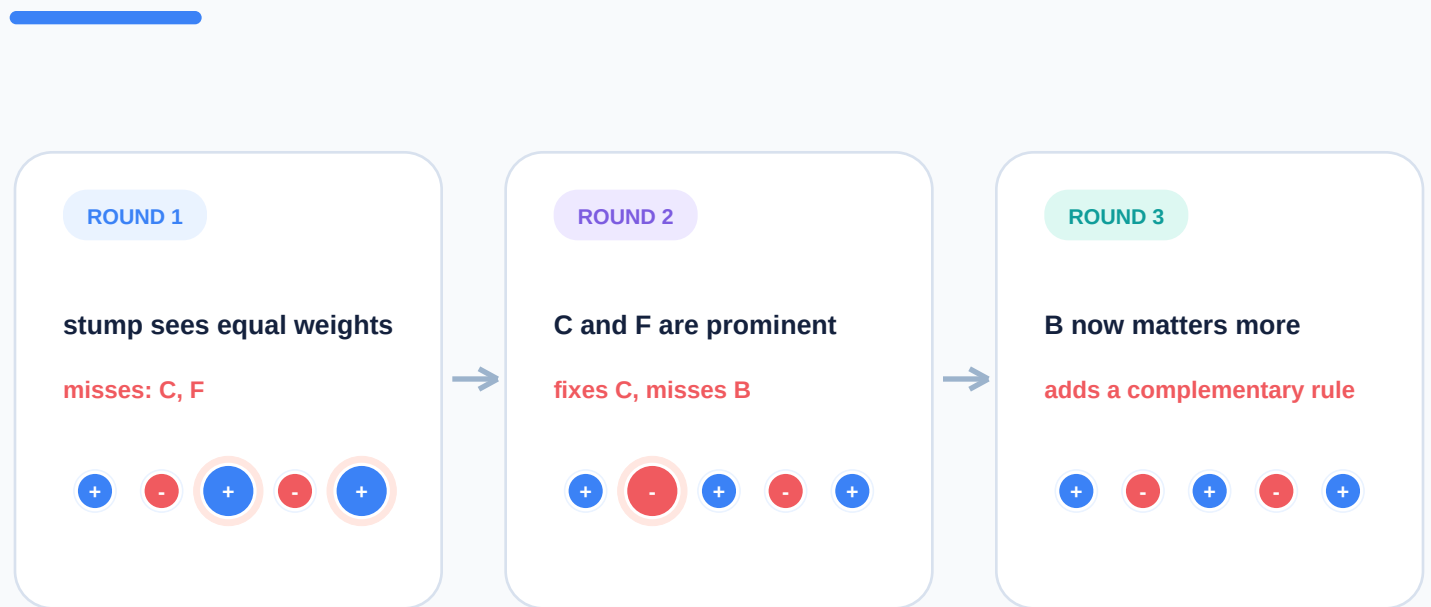
The learner sees difficult cases again and again. Easy cases still appear, but less often.

Same data.

Different emphasis.

07 The full boosting journey

Each round changes the dataset the next stump effectively sees.



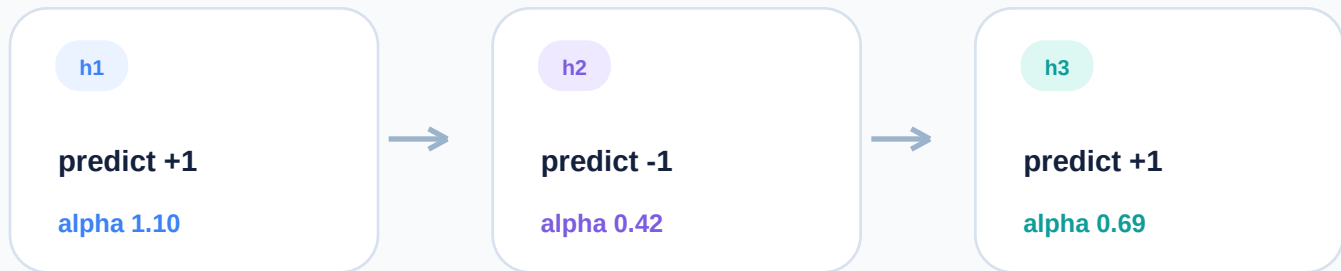
Iteration checklist

- 1 Fit $h(t)$ using current weights
- 2 Measure weighted error $e(t)$
- 3 Set $\alpha(t)$ from $e(t)$
- 4 Increase missed-example weights
- 5 Normalize and repeat

The outcome is an accumulating committee of learners, not a chain of replacements.

08 Final prediction: weighted vote

Every stump speaks; alpha decides how loudly each one speaks.



$$F(x) = \text{sign}(\text{sum } \alpha(t) h(t)(x))$$

For this example: $(+1 \times 1.10) + (-1 \times 0.42) + (+1 \times 0.69) = +1.37$

The vote meter

Final class: +1



Use this page to rehearse the order of operations before coding AdaBoost.

Micro example

N = 8

initial $w = 1/8 = .125$

stump misses 2 rows

$e = .125 + .125 = .25$

compute strength

$\alpha = .5 \ln(.75/.25) = .55$

update

wrong rows get multiplied by $\exp(+.55)$

Three checks before next round

-  Are the weights normalized to 1?
-  Is the learner better than random?
-  Do the high-weight misses make sense?

AdaBoost in five lines

```
initialize  $w(i) = 1 / N$ 
```

```
for each round  $t$ :
```

```
    fit weak learner  $h(t)$  using  $w$ 
```

```
    compute  $e(t)$ , then  $\alpha(t)$ 
```

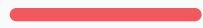
```
    upweight mistakes; normalize; vote at the end
```

Remember: α measures a learner; weights measure examples.

References: Freund & Schapire (1997), Hastie et al. (2009), scikit-learn AdaBoost documentation.

One complete round - with numbers

Pause at each card, predict what changes, then trace the arrows.



1. START

8 rows, each $w = .125$

2. STUMP

misses rows C and F

3. SCORE

$e = .25$, $\alpha = .55$

4. UPDATE

C and F receive more mass

Weight ledger

row	before	correct?	after normalize
A	0.125	yes	0.083
B	0.125	yes	0.083
C	0.125	NO	0.251
D	0.125	yes	0.083
E	0.125	yes	0.083
F	0.125	NO	0.251
G	0.125	yes	0.083
H	0.125	yes	0.083

Why the math works

The two errors go from 25% of total attention to just over 50% together.

A later stump is now strongly encouraged to fix C and F.

Try it

If the same stump misses C and F again, should its alpha grow or shrink?

Answer: shrink. Those errors now carry much more weight, so the same rule has a worse weighted error.

AdaBoost is powerful when weak rules complement one another - and fragile when noise dominates.

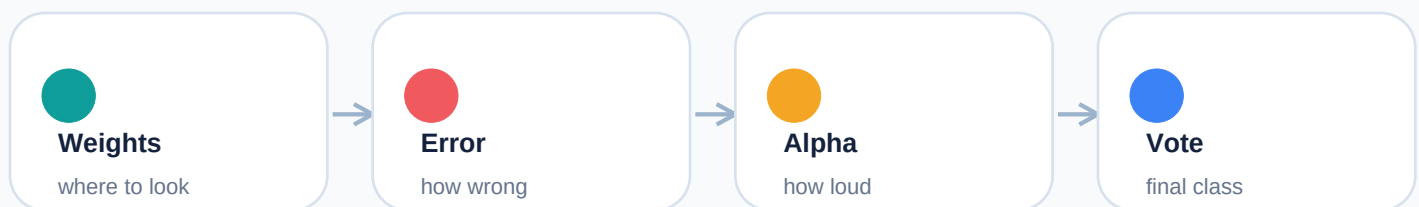
A good fit

- + Clean signal**
Simple patterns can combine.
- + Useful features**
Stumps can split meaningful columns.
- + Moderate noise**
Hard examples are not mostly label errors.
- + Fast baseline**
You want interpretable learners quickly.

Watch out for

- ! Label noise**
The model may chase incorrect labels.
- ! Outliers**
Repeated focus can over-amplify them.
- ! Weak learner too strong**
You lose the intended gradual correction.
- ! Error near .50**
The learner contributes almost no value.

Fast recall map



If you remember one sentence: AdaBoost keeps asking, "Which mistakes deserve more attention next?"

Use the numerical round on page 10 once before you implement it.